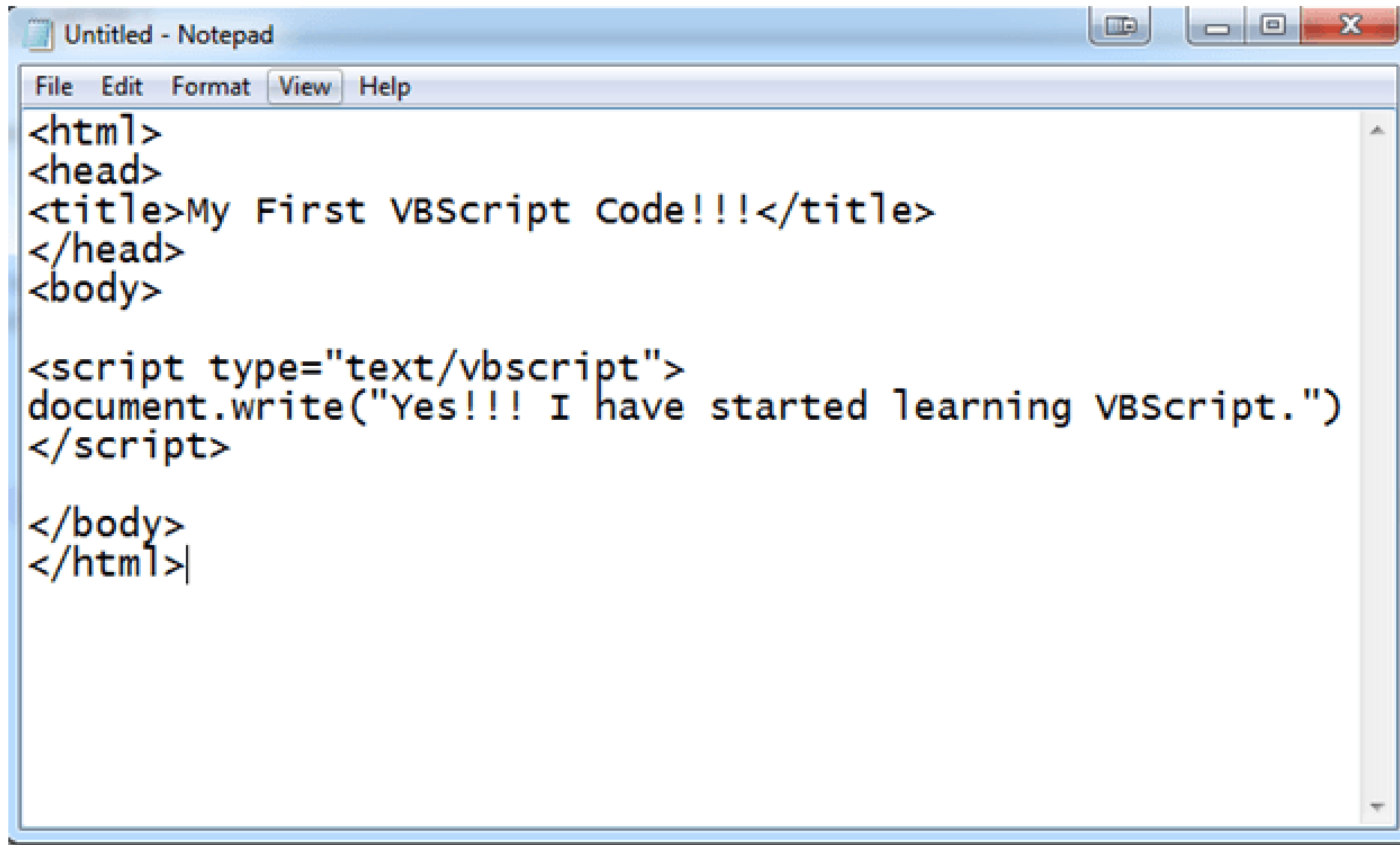


Virtualization-Based (In)security Weaponizing VBS Enclaves

Ori David





The image shows a standard Windows Notepad application window. The title bar at the top reads "Untitled - Notepad". Below the title bar is a menu bar with the options "File", "Edit", "Format", "View", and "Help". The main text area contains the following code:

```
<html>
<head>
<title>My First VBScript Code!!!</title>
</head>
<body>

<script type="text/vbscript">
document.write("Yes!!! I have started learning VBScript.")
</script>

</body>
</html>|
```

The code is a simple HTML document structure. It includes a head section with a title "My First VBScript Code!!!". The body section contains a VBScript code block that uses the `document.write` method to output the text "Yes!!! I have started learning VBScript." to the browser. The cursor is positioned at the end of the last line of the HTML document, after the closing `</html>` tag.

A man in a blue plaid shirt is walking away from a woman in a red dress on a city street. He is looking back over his shoulder at her. The woman is smiling. The background is a blurred city street with other people.

**VIRTUALIZATION
BASED SECURITY**

**VISUAL
BASIC SCRIPT**

Whoami

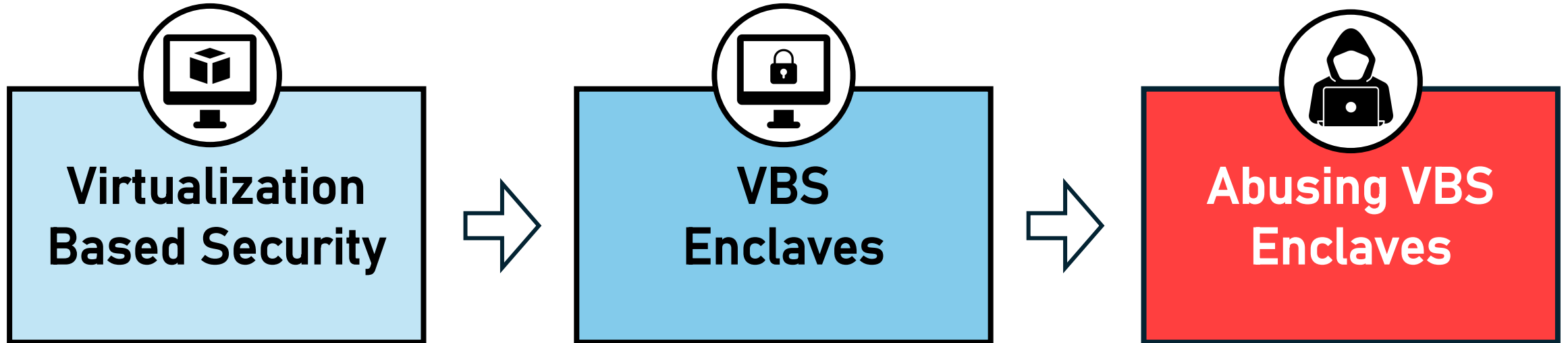
Ori David

Security researcher at Akamai

Background in red teaming and threat hunting



Agenda

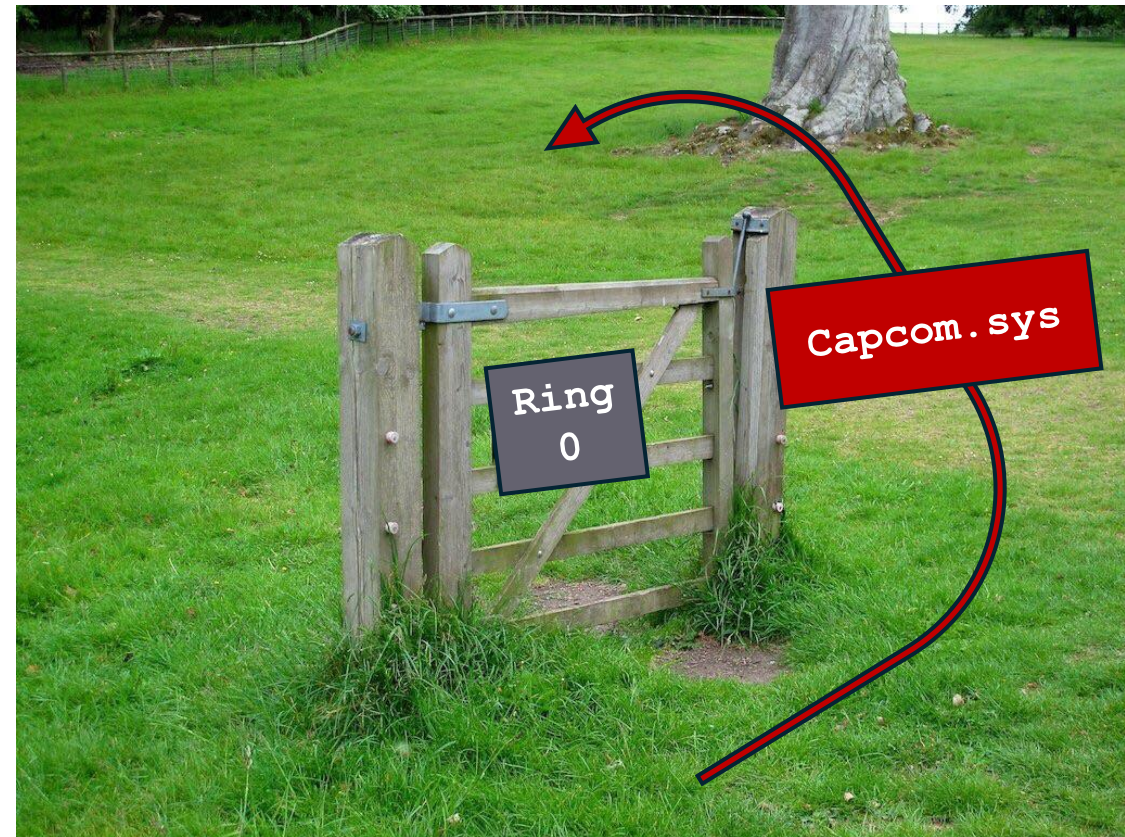




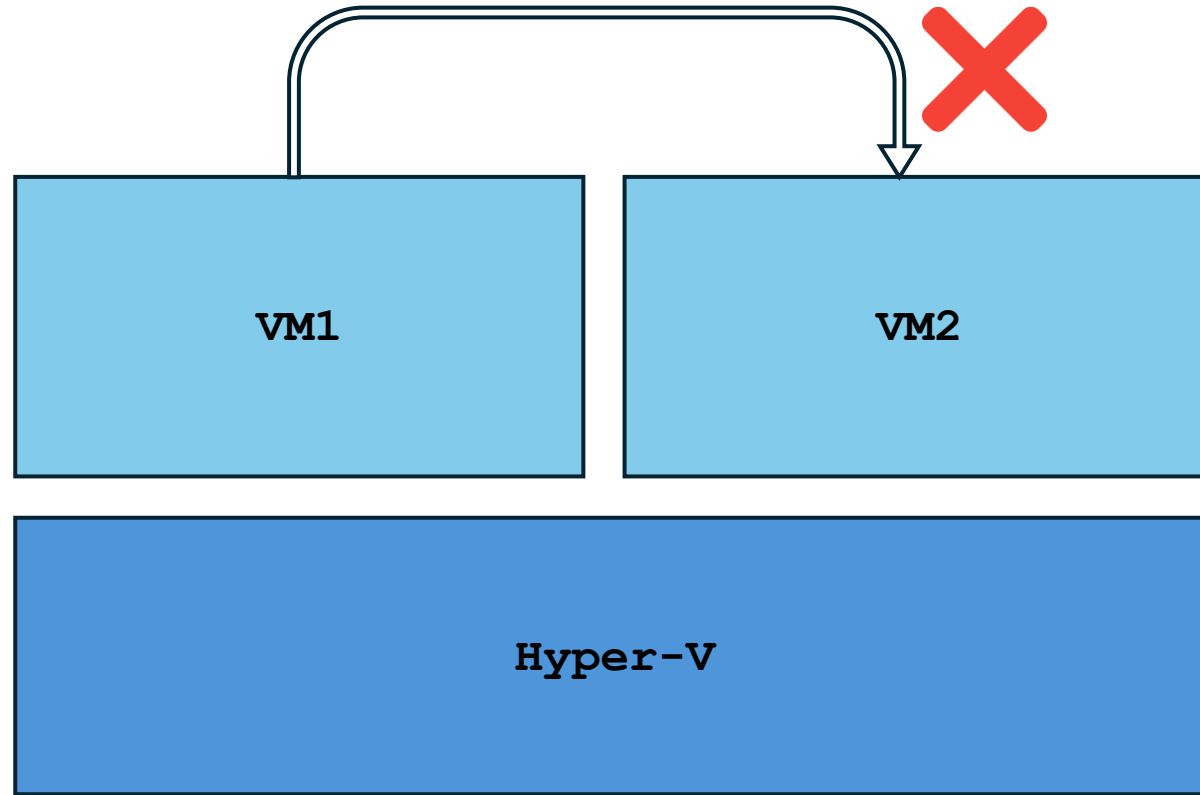
Virtualization-Based Security

Pushing the boundaries

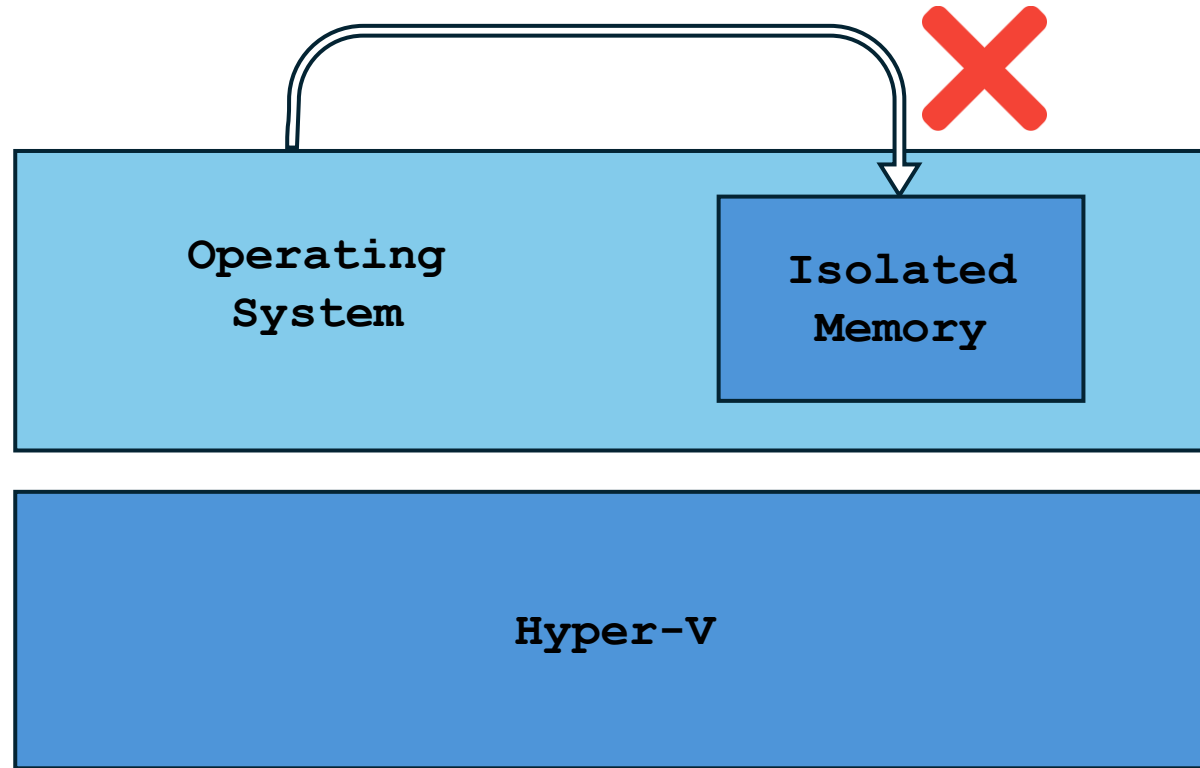
- Windows traditionally relied on the ring3/0 boundary
- This proved to be insufficient
 - Kernel exploits
 - Bring Your Own Vulnerable Driver



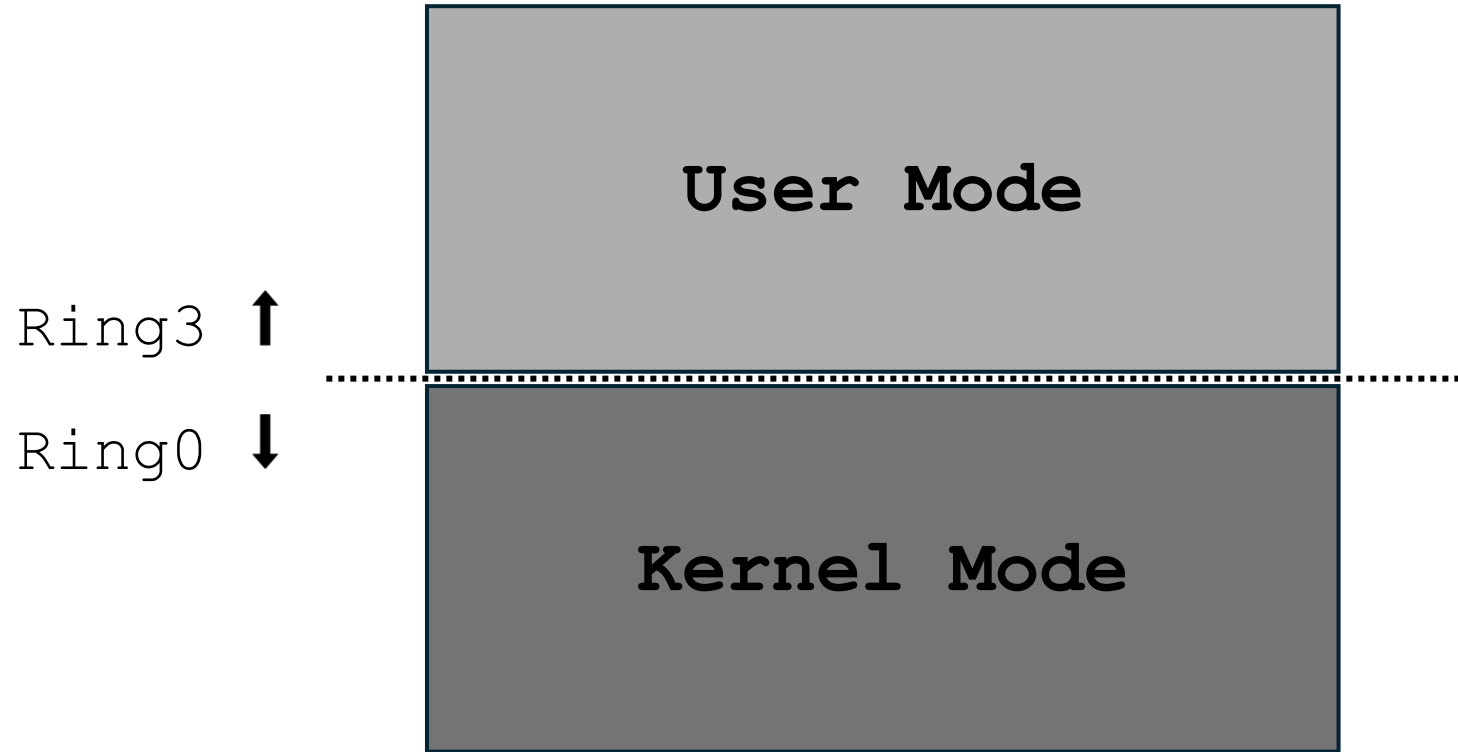
Virtualization Based Security



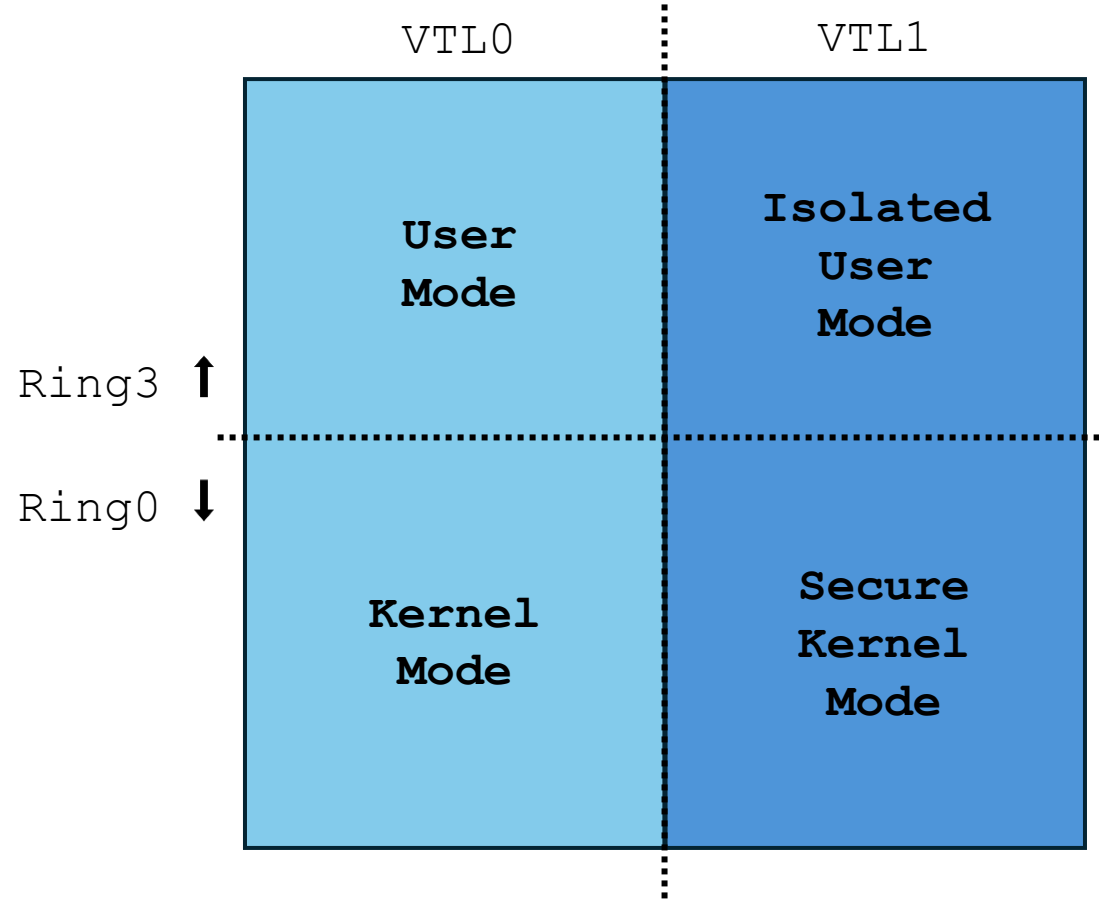
Virtualization Based Security



Virtual Trust Level (VTL)



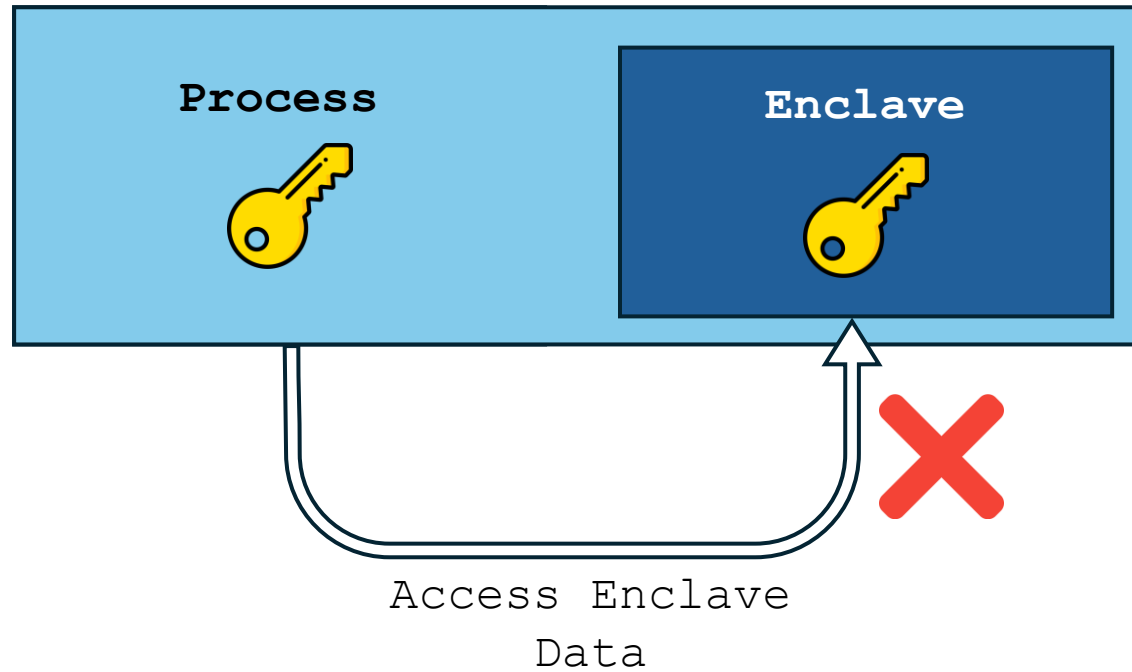
Virtual Trust Level (VTL)



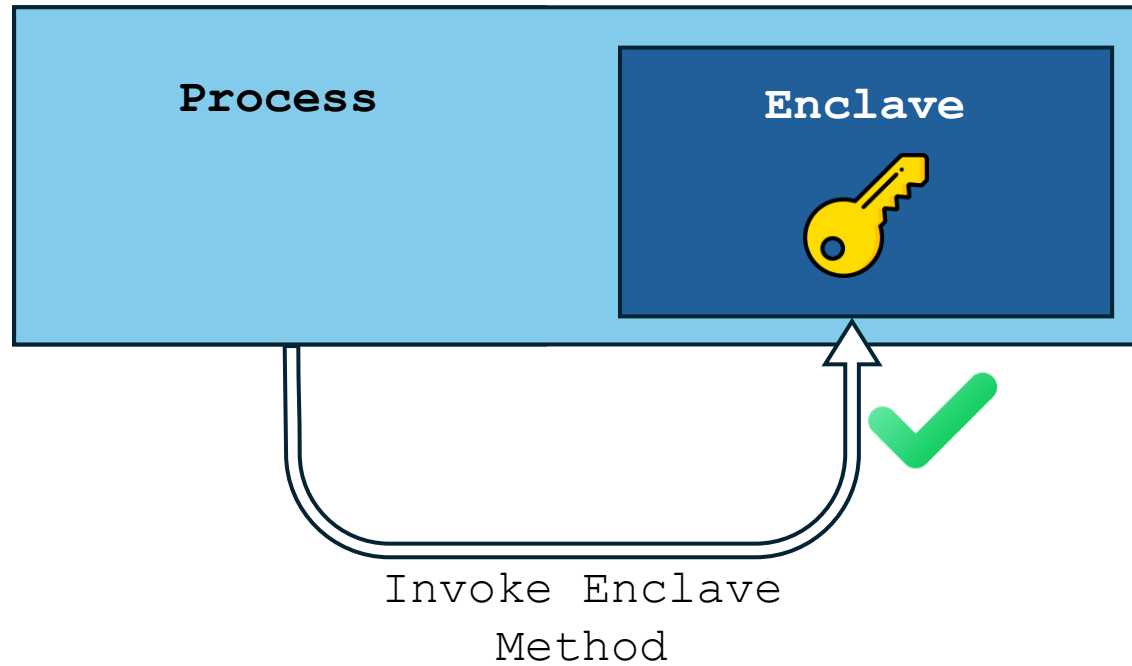


VBS Enclaves

What is an Enclave?

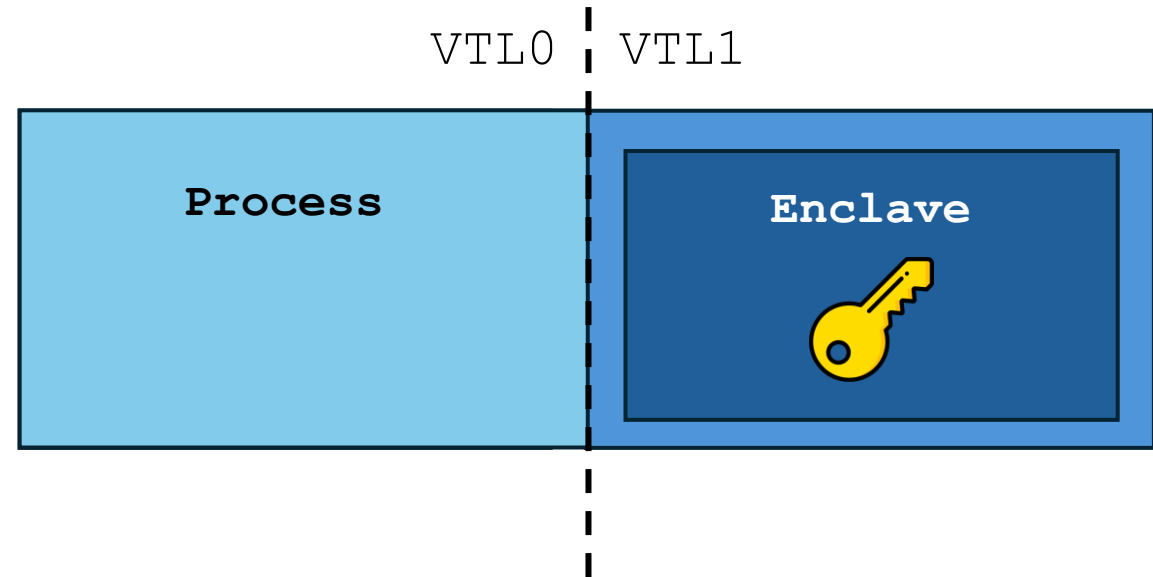


What is an Enclave?

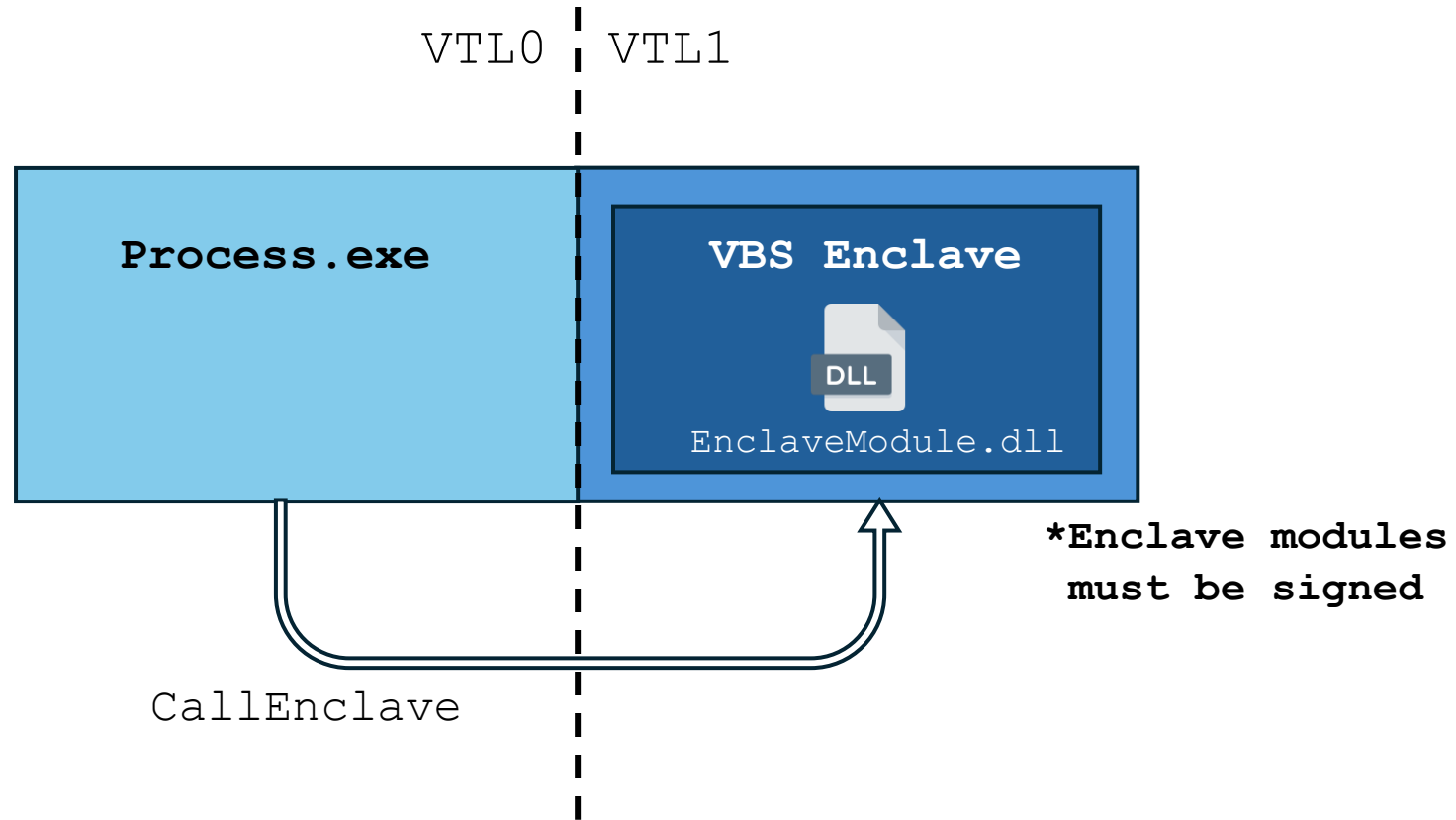


VBS Enclaves

- Execute part of a process in IUM
- Nothing in VTL0 can access the enclave



VBS Enclave Lifecycle





Abusing VBS Enclaves

Enclave Abuse Potential

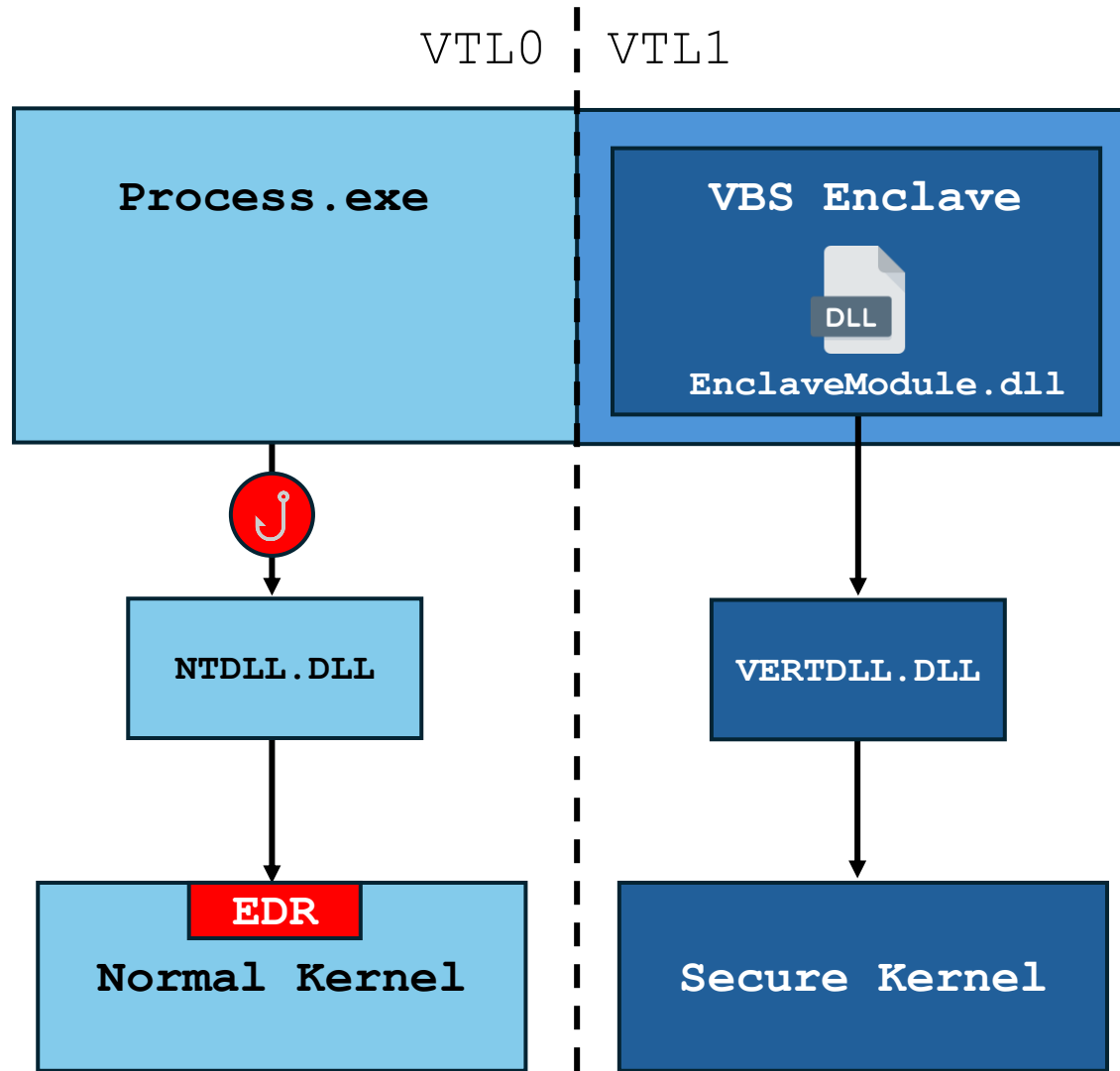
Inaccessible Memory



Untraceable API calls



VBS Enclave API Calling



Loading a Malicious Enclave



Obtain a Legitimate Signature

- Enclave signing is exposed to 3rd parties
 - Compromise authorized signer
 - Obtain signing privileges legitimately



Exploit a Loader Vulnerability

- Exploit a vulnerability in the enclave loading process
 - CVE-2024-49076 by Alex Ionescu

What privileges would an attacker gain by successfully exploiting this vulnerability?

An attacker who successfully exploited this vulnerability could load a non-Microsoft DLL into an enclave, potentially leading to code execution within the context of the target enclave.



Abusing Debuggable Enclave Modules

Debuggable Enclave Modules

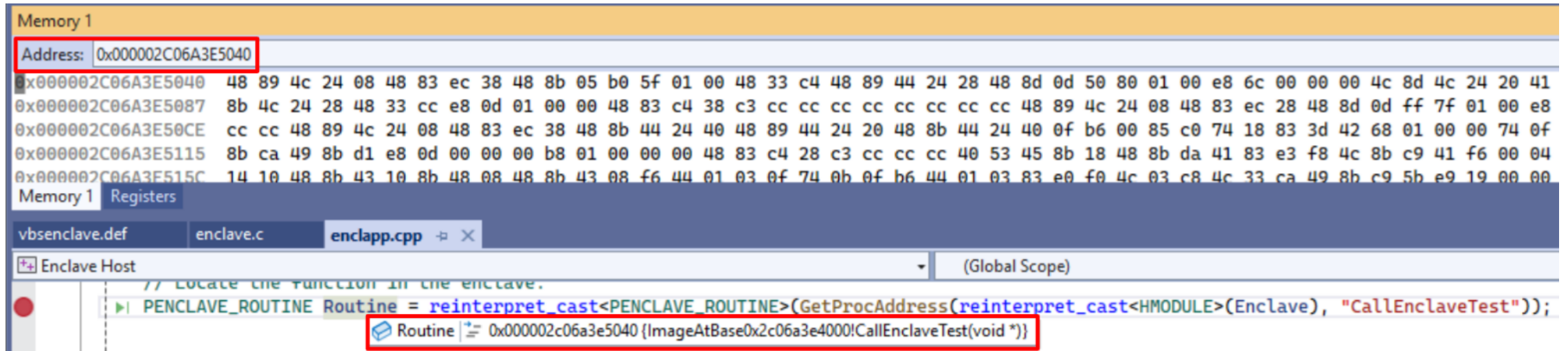
- As enclaves reside in VTL1, debugging them isn't straightforward

The screenshot displays a debugger interface with the following components:

- Memory 1:** A panel showing memory addresses and their corresponding hex values. The address `0x000001C5A2BE5040` is highlighted with a red box. The values are mostly `??`, indicating unknown or debugged data.
- Registers:** A panel showing the state of registers, currently empty.
- Code:** A panel showing the source code of the enclave module. The file `enclave.c` is selected. The code is in C and shows a function `CallEnclaveTest` that calls `GetProcAddress` to find the `CallEnclaveTest` function in the enclave. The line `PENCLAVE_ROUTINE Routine = reinterpret_cast<PENCLAVE_ROUTINE>(GetProcAddress(reinterpret_cast<HMODULE>(Enclave), "CallEnclaveTest"));` is highlighted. Below the code, a red box highlights the routine `Routine 0x000001c5a2be5040`.

Debuggable Enclave Modules

- To solve this, we can create debuggable enclave modules



Memory 1

Address: 0x000002C06A3E5040

0x000002C06A3E5040 48 89 4c 24 08 48 83 ec 38 48 8b 05 b0 5f 01 00 48 33 c4 48 89 44 24 28 48 8d 0d 50 80 01 00 e8 6c 00 00 00 4c 8d 4c 24 20 41

0x000002C06A3E5087 8b 4c 24 28 48 33 cc e8 0d 01 00 00 48 83 c4 38 c3 cc cc cc cc cc cc 48 89 4c 24 08 48 83 ec 28 48 8d 0d ff 7f 01 00 e8

0x000002C06A3E50CE cc cc 48 89 4c 24 08 48 83 ec 38 48 8b 44 24 40 48 89 44 24 20 48 8b 44 24 40 0f b6 00 85 c0 74 18 83 3d 42 68 01 00 00 74 0f

0x000002C06A3E5115 8b ca 49 8b d1 e8 0d 00 00 00 b8 01 00 00 00 48 83 c4 28 c3 cc cc cc 40 53 45 8b 18 48 8b da 41 83 e3 f8 4c 8b c9 41 f6 00 04

0x000002C06A3E515C 14 10 48 8b 43 10 8b 48 08 48 8b 43 08 f6 44 01 03 0f 74 0b 0f b6 44 01 03 83 e0 f0 4c 03 c8 4c 33 ca 49 8b c9 5b e9 19 00 00

Memory 1 Registers

vbsenclave.def enclave.c enclapp.cpp

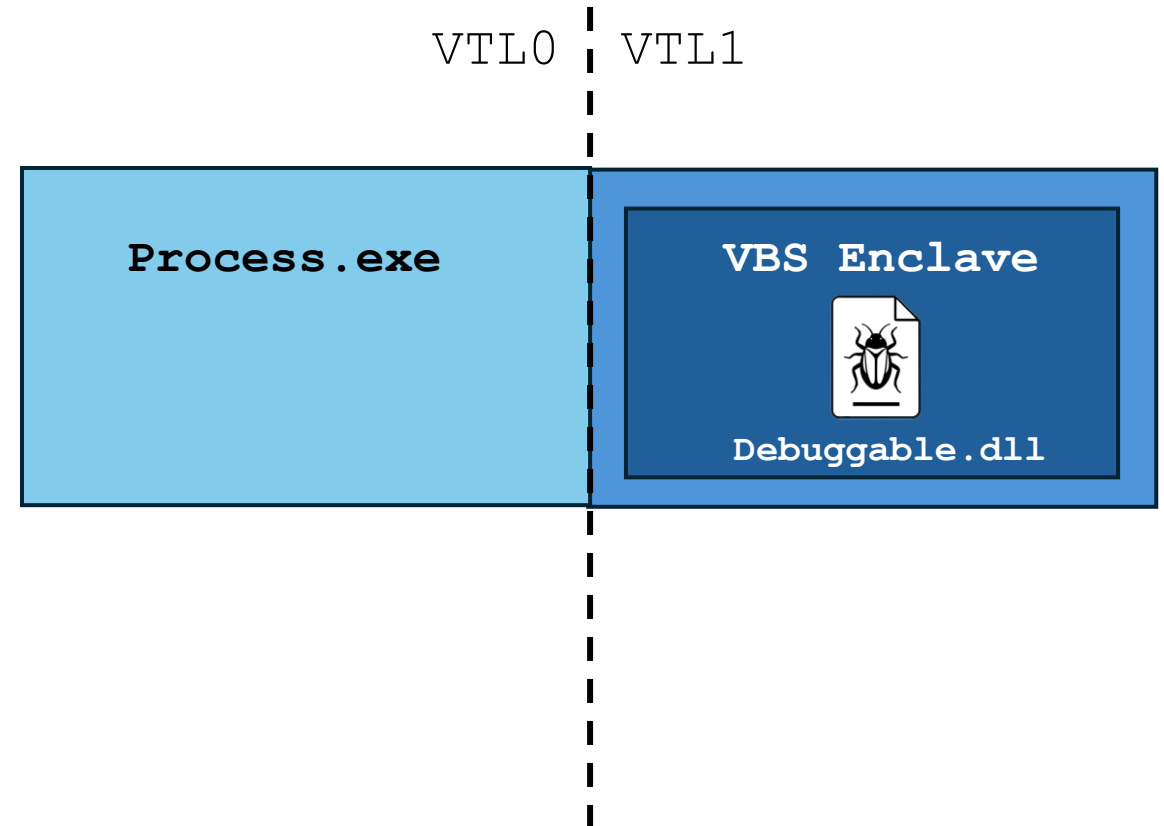
Enclave Host (Global Scope)

```
// Locate the function in the enclave.  
PENCLAVE_ROUTINE Routine = reinterpret_cast<PENCLAVE_ROUTINE>(GetProcAddress(reinterpret_cast<HMODULE>(Enclave), "CallEnclaveTest"));
```

Routine = 0x000002c06a3e5040 {ImageAtBase0x2c06a3e4000!CallEnclaveTest(void *)}

Debuggable Enclave Modules

- Interestingly, debuggable enclaves reside in IUM
- To enable debugging, the secure kernel implements exceptions
 - *SkmmDebugReadWriteMemory*
 - *SkmmDebugProtectVirtualMemory*



Abusing Debuggable Enclaves

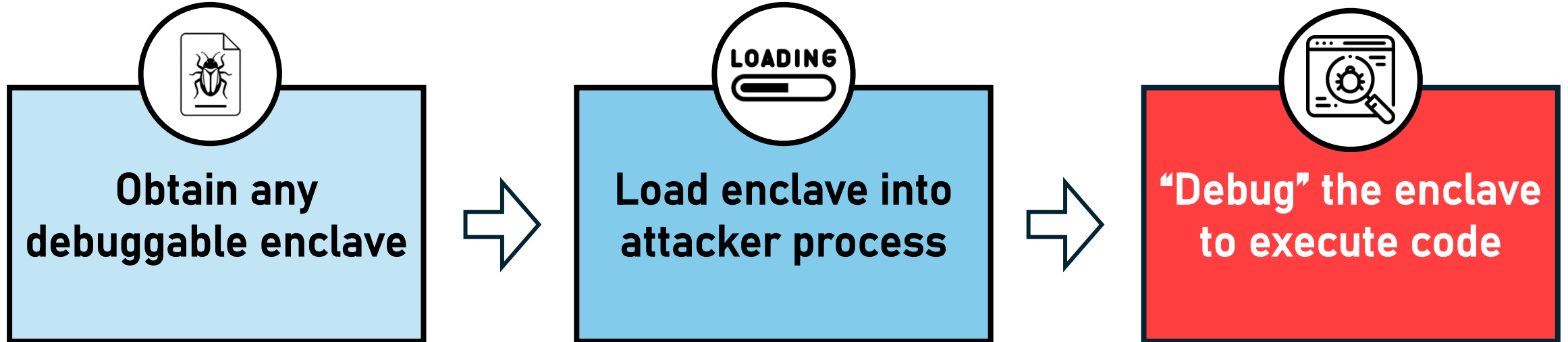
- Obviously, using a debuggable enclave in production is not a good idea

ⓘ Note

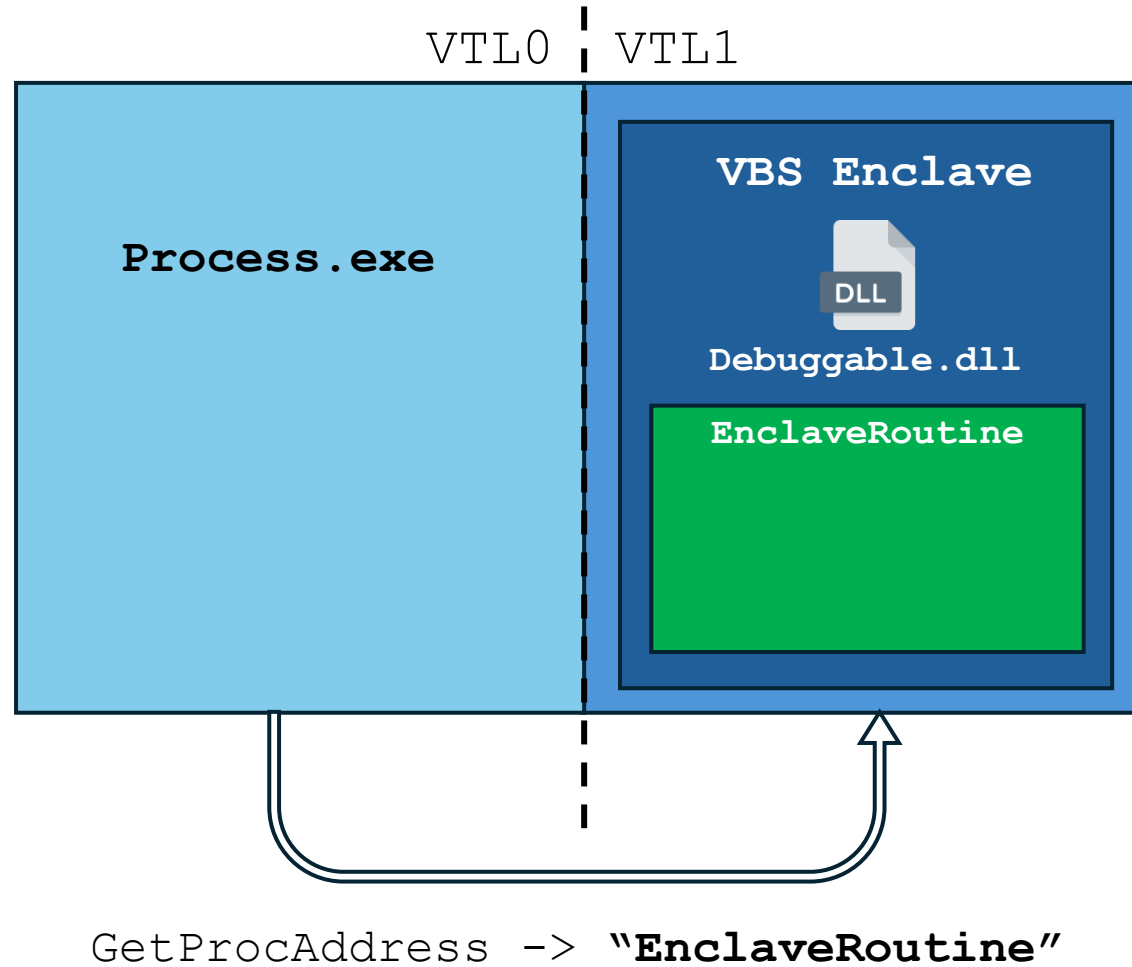
Never use debuggable enclaves in production environments.

- But that's not the only problem!

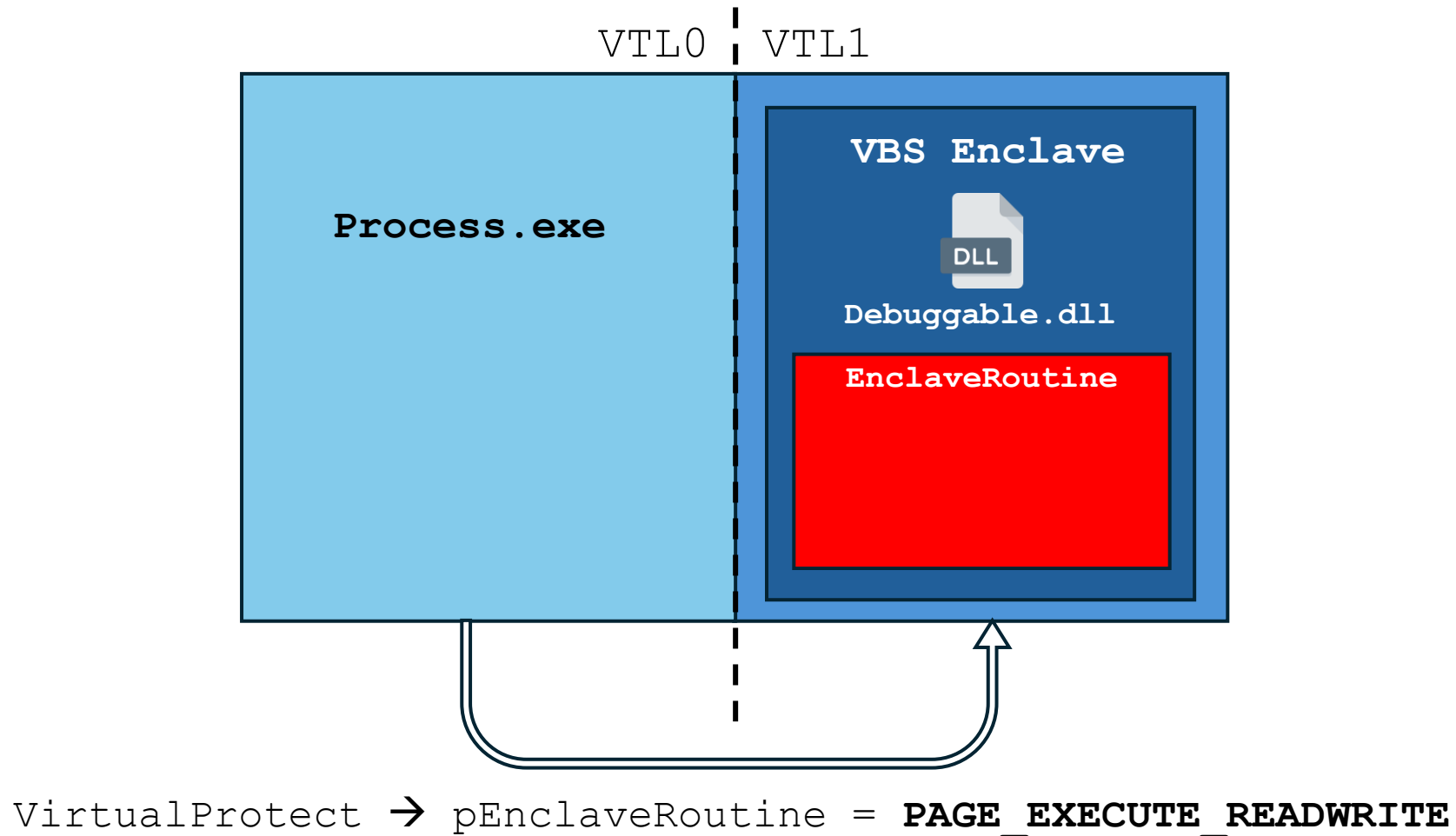
Abusing Debuggable Enclaves



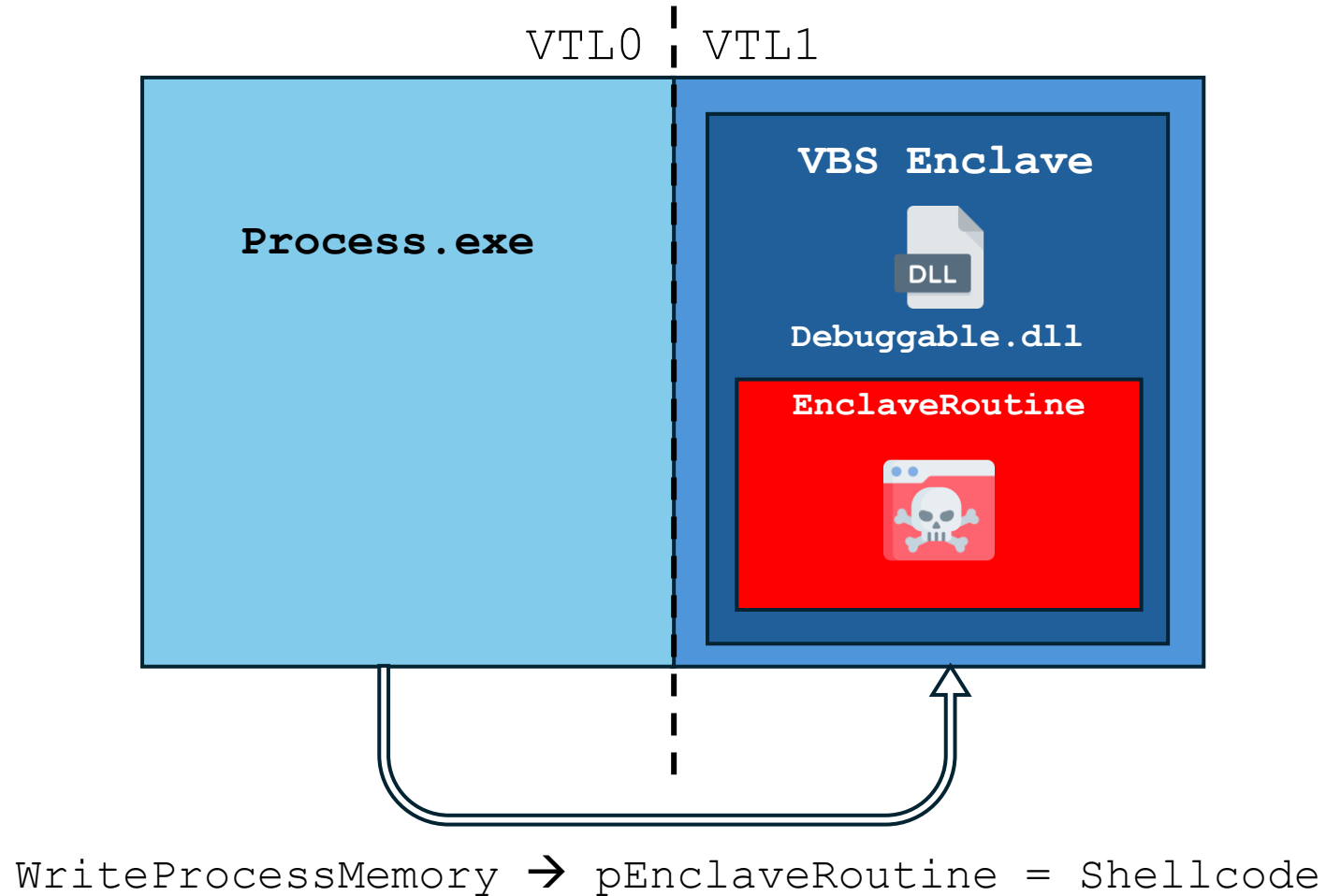
Abusing Debuggable Enclaves to Execute Code in IUM



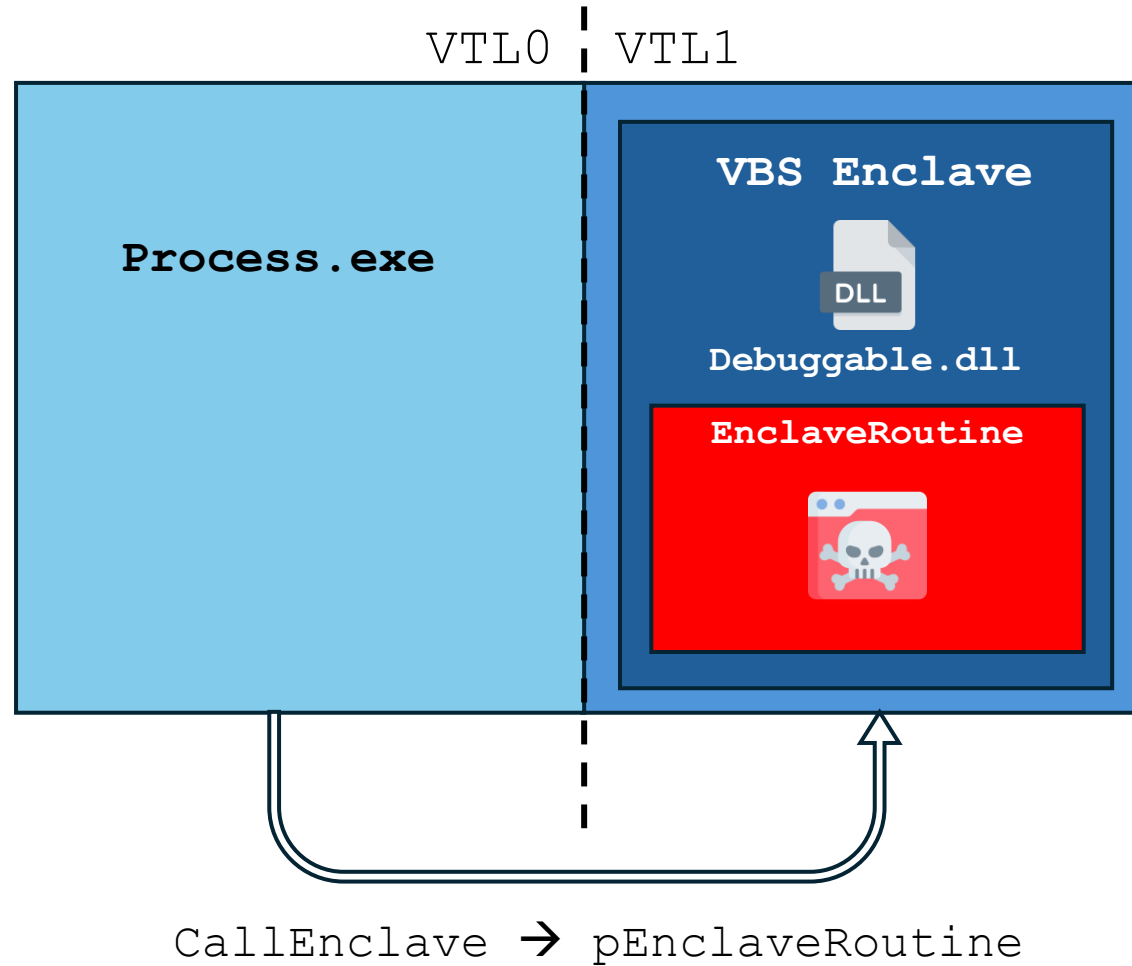
Abusing Debuggable Enclaves to Execute Code in IUM



Abusing Debuggable Enclaves to Execute Code in IUM



Abusing Debuggable Enclaves to Execute Code in IUM



Abusing Debuggable Enclaves – The Catch

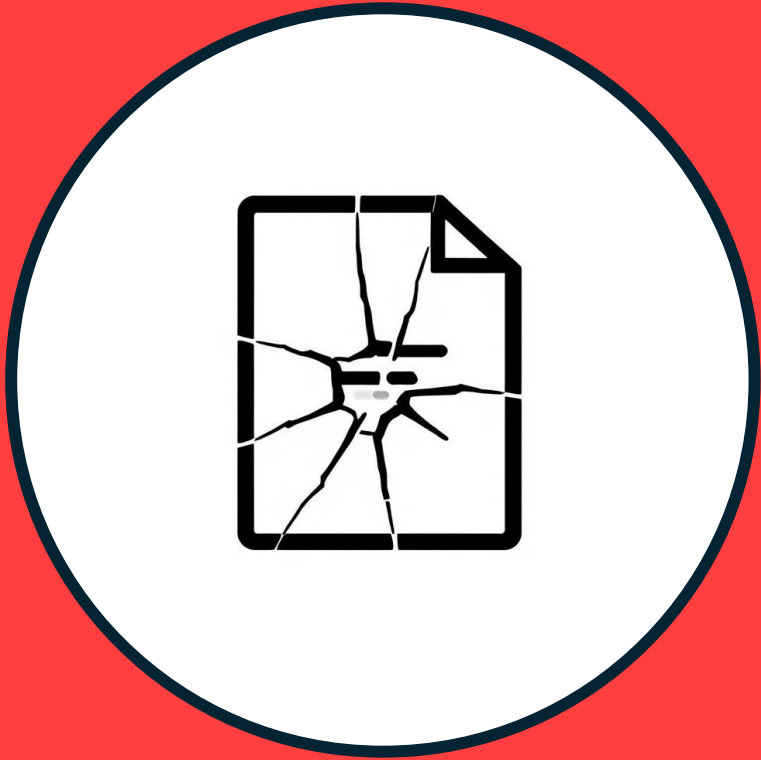
- The unrestricted enclave memory access goes both ways
- Despite that – there are still advantages:
 - An EDR/analyst might not consider this scenario
 - Enclave code still evades VTLO API monitoring



Abusing Debuggable Enclaves – ~~HELP~~ ME

- I haven't managed to find one yet, but some will likely leak eventually





Exploiting Vulnerable Enclave Modules

BYONE



Bring
your own
vulnerable driver



Bring your
own
vulnerable enclave

CVE-2023-36880

- Information disclosure vulnerability in a Microsoft Edge enclave module
- Could allow limited code execution(?)

According to the CVSS metric, successful exploitation of this vulnerability could lead to some loss of integrity (I:L). What does that mean for this vulnerability?

The attacker who successfully exploited the vulnerability could have limited ability to perform code execution

CVE-2023-36880

- Discovered by Alex Gough, who also shared a PoC
- The vulnerability provides an arbitrary read/write primitive within the enclave

Microsoft Edge: Arbitrary Perms

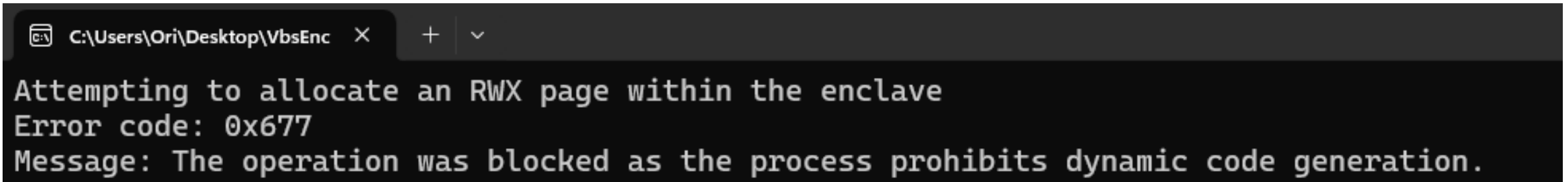
Moderate

rcorrea35 published GHSA-wwr4-v5mr-3x9w on Dec 14, 2023

Package	Affected versions
Edge (Microsoft)	< https://msrc.microsoft.com/update-guide/vulnerability/CVE-2023-36880

Exploiting CVE-2023-36880

- Enclaves are protected with Arbitrary Code Guard (ACG) by default
 - No new executable pages during runtime
 - Cannot turn executable pages writable



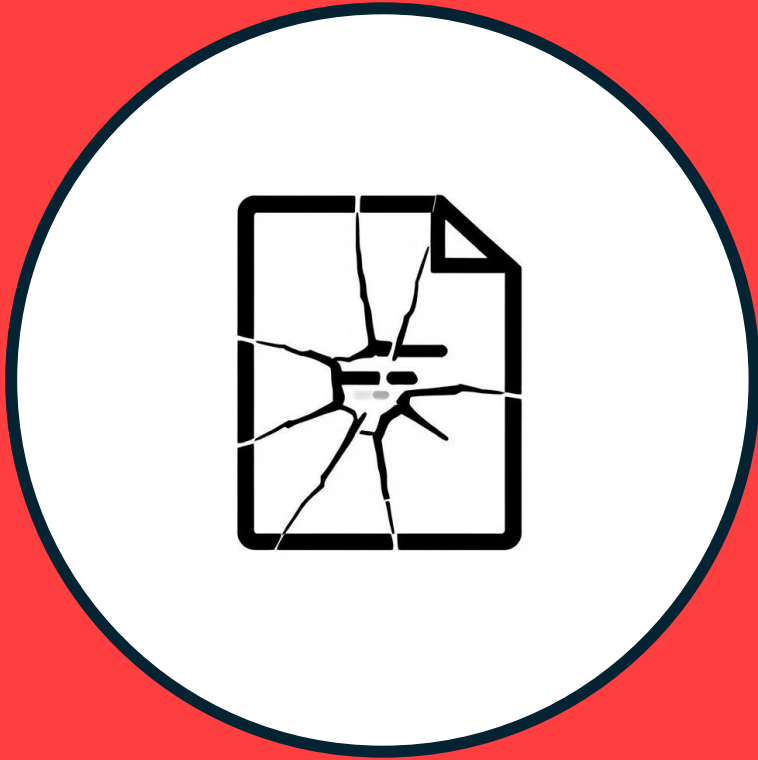
A screenshot of a Windows command prompt window. The title bar shows the file path 'C:\Users\Ori\Desktop\VbsEnc' and standard window controls. The command prompt displays the following text:

```
Attempting to allocate an RWX page within the enclave  
Error code: 0x677  
Message: The operation was blocked as the process prohibits dynamic code generation.
```




He's thinking of other women

A RW primitive within an
enclave got to be useful somehow



Bring Your Own Vulnerable Enclave

- Round 2 -

CVE-2023-36880 – What **can** we do?

Write data into VTL1



**Store data out of the
reach of EDRs**

**Write data into VTL0
from VTL1**



**Modify data within the
process while
bypassing hooks**

Mirage – VTL1 Based Memory Evasion

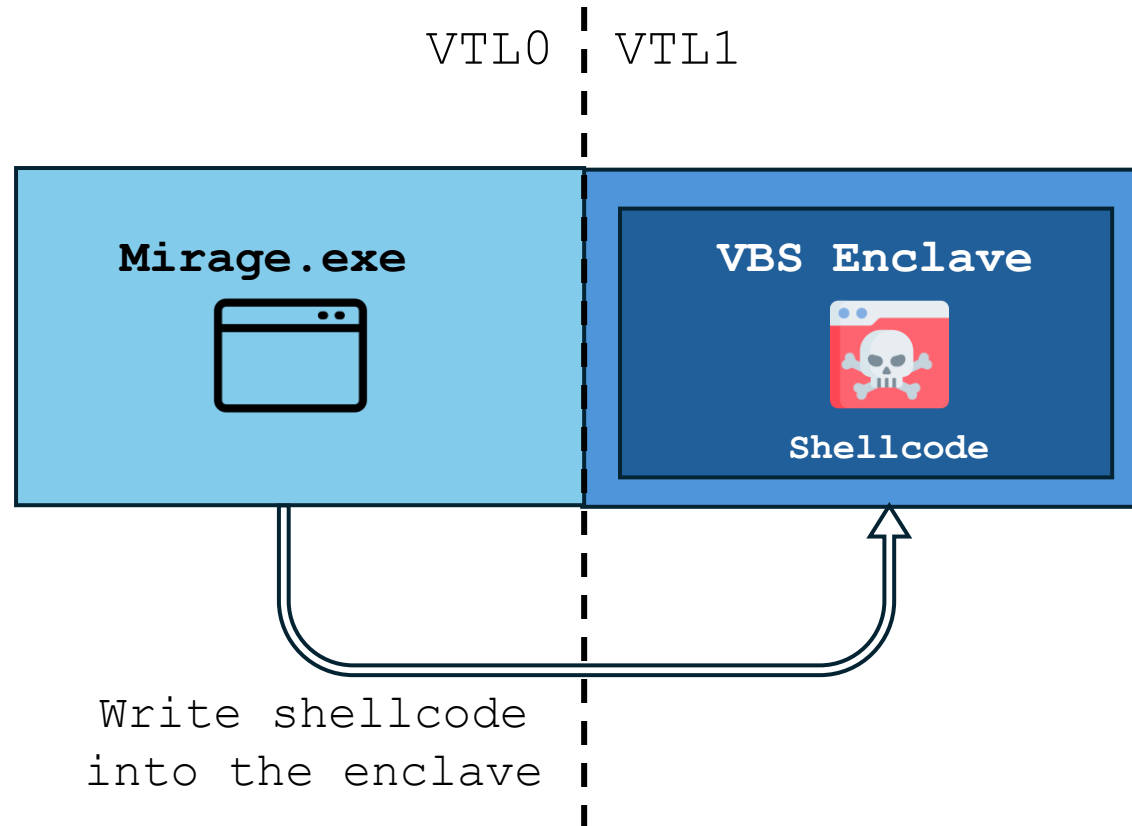
- Sleep obfuscation technique
- Hide shellcode in IUM during sleep

```
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

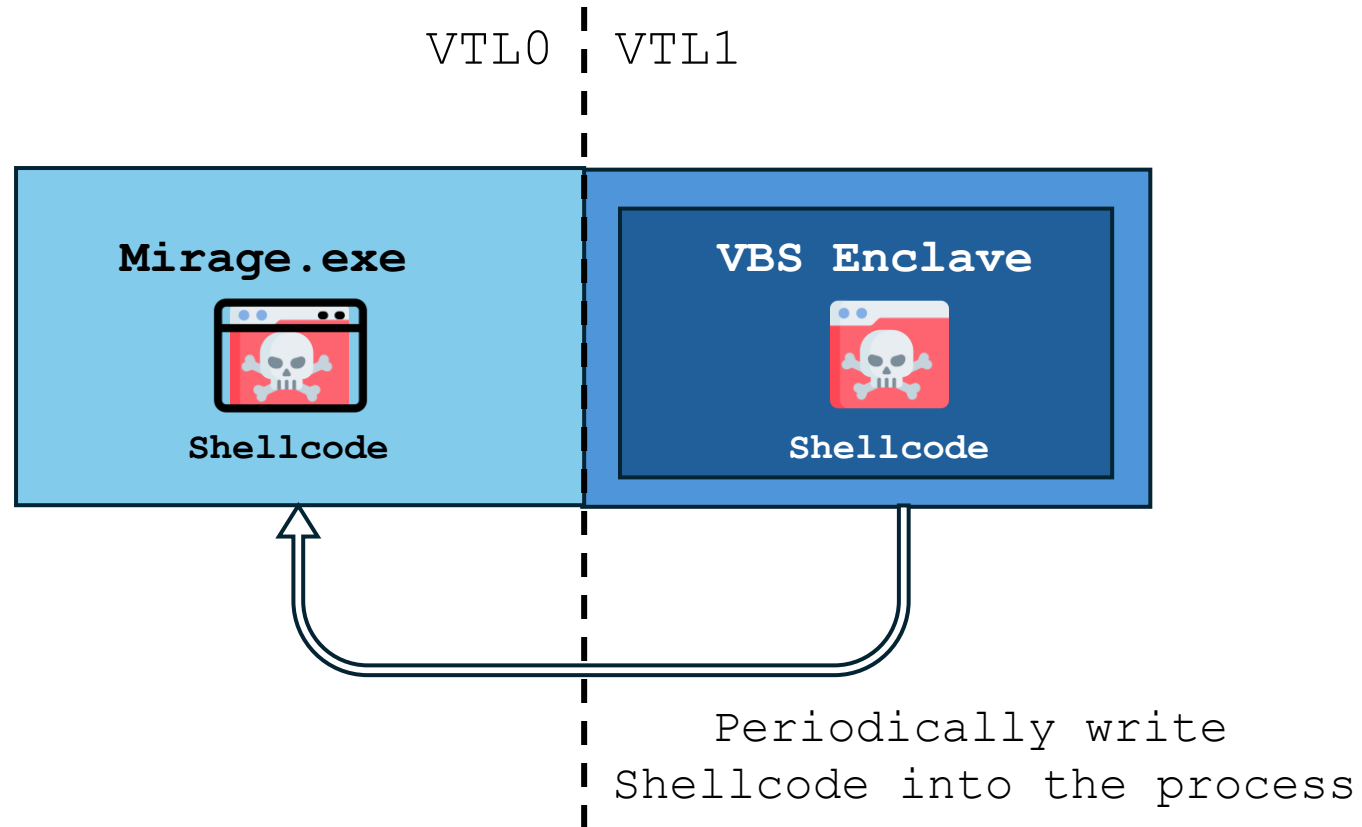


```
49 89 E7 48 31 FF 48 F7 E7 65 48 8B 58 60 48 8B
5B 18 48 8B 5B 20 48 8B 1B 48 8B 1B 48 8B 5B 20
49 89 D8 8B 5B 3C 4C 01 C3 48 31 C9 66 81 C1 FF
88 48 C1 E9 08 8B 14 0B 4C 01 C2 4D 31 D2 44 8B
52 1C 4D 01 C2 4D 31 DB 44 8B 5A 20 4D 01 C3 4D
31 E4 44 8B 62 24 4D 01 C4 EB 32 5B 59 48 31 C0
48 89 E2 51 48 8B 0C 24 48 31 FF 41 8B 3C 83 4C
01 C7 48 89 D6 F3 A6 74 05 48 FF C0 EB E6 59 66
41 8B 04 44 41 8B 04 82 4C 01 C0 53 C3 48 31 C9
80 C1 07 48 B8 0F A8 96 91 BA 87 9A 9C 48 F7 D0
48 C1 E8 08 50 51 E8 B0 FF FF FF 49 89 C6 48 31
C9 48 F7 E1 50 48 B8 9C 9E 93 9C D1 9A 87 9A 48
F7 D0 50 48 89 E1 48 FF C2 48 83 EC 20 41 FF D6
4C 89 FC C3 6D 69 6D 69 6B 61 74 7A 2E 65 78 65
```

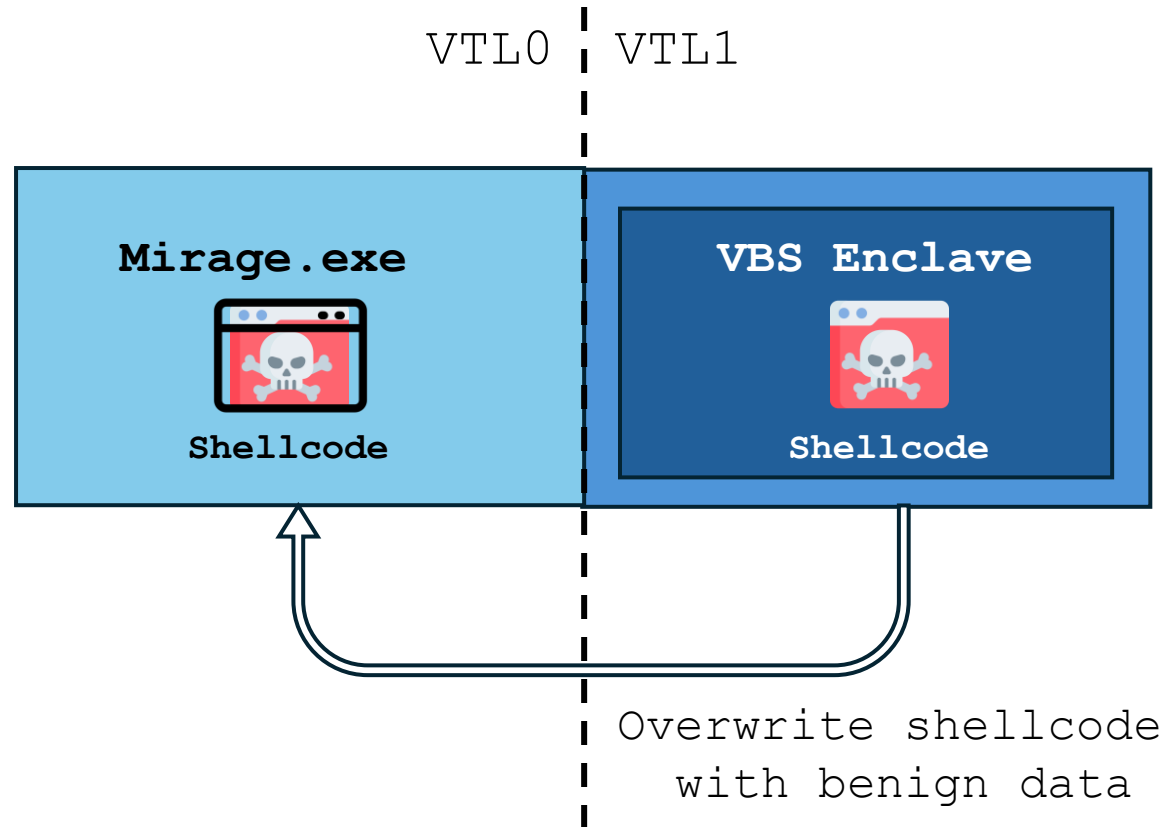
Mirage – VTL1 Based Memory Evasion



Mirage – VTL1 Based Memory Evasion



Mirage – VTL1 Based Memory Evasion

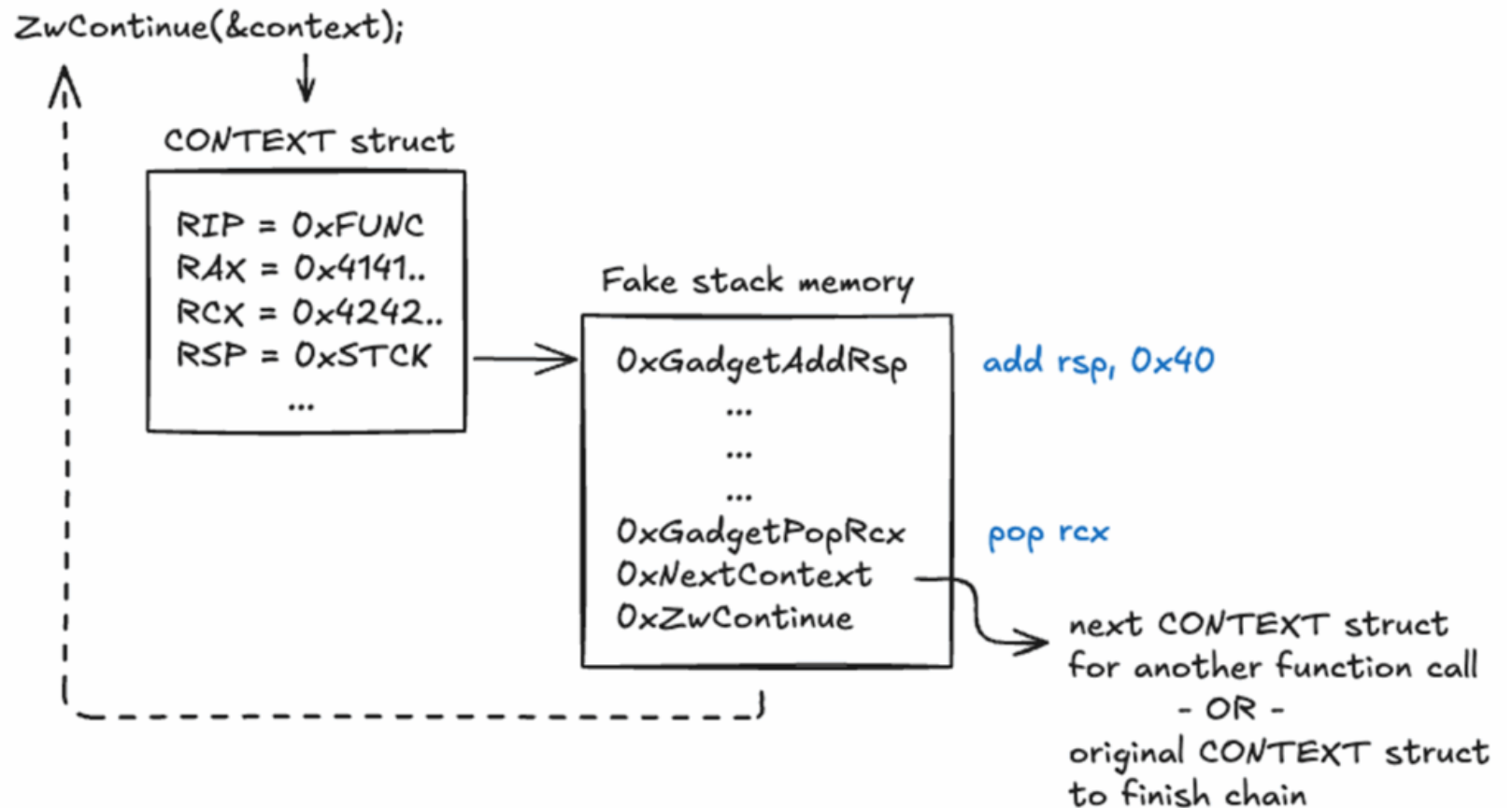


Mirage - Demo



BYOME - Round 1.5

- Cedric Van Bockhaven and Matteo Malvica of Outflank demonstrated a full ROP exploit to achieve VTL1 RCE



Enclave

Malware

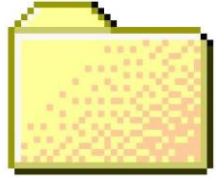
TTPs

Protecting Secrets

- Use the enclave as it was intended to – hide secrets
 - Additional payloads
 - Encryption keys
 - Configuration



Things We **Can't** Do



**File
Operations**



Networking



**Process
Interaction**



**Registry
Access**

Things We **Can** Do

???

Access VTL0 Memory

- Enclaves have read/write access to the VTL0 usermode memory of the process



Access VTL0 Memory

- Access adheres to memory protection permissions

nên chỉ có thể truy cập địa chỉ địa phương của nó

nên chỉ có thể truy cập địa chỉ địa phương của nó

- Cannot change VTL0 memory protection permissions

Chỉ có thể truy cập địa chỉ địa phương của nó RAGÉ READWRITE ô địa

Monitor VTL0 Memory

chấs cấđ ṣṭṣịṇg̣ ΛB\$ ịṣ lặ́ṇê
chấs gồồđ ṣṭṣịṇg̣ ΛB\$ ịṣ CỒÔL'

xḥịḷê ,

ịg̣ ṣṭṣç̣ṇṛ ẉṭḷ. c̣ụg̣g̣ệṣ cấđ ṣṭṣịṇg̣ .

ṇệṇç̣ṛỵ ẉṭḷ. c̣ụg̣g̣ệṣ gồồđ ṣṭṣịṇg̣ ṣṭṣḷêṇ gồồđ ṣṭṣịṇg̣

Execute VTL0 code

- Enclaves cannot execute VTL0 code within VTL1

injt ġuŋç injt êyêçuŧǎčlê wŧl. ǎđđsêşş
injt sêşu'ŧ ġuŋç

- But – they can invoke VTL0 code remotely via CallEnclave

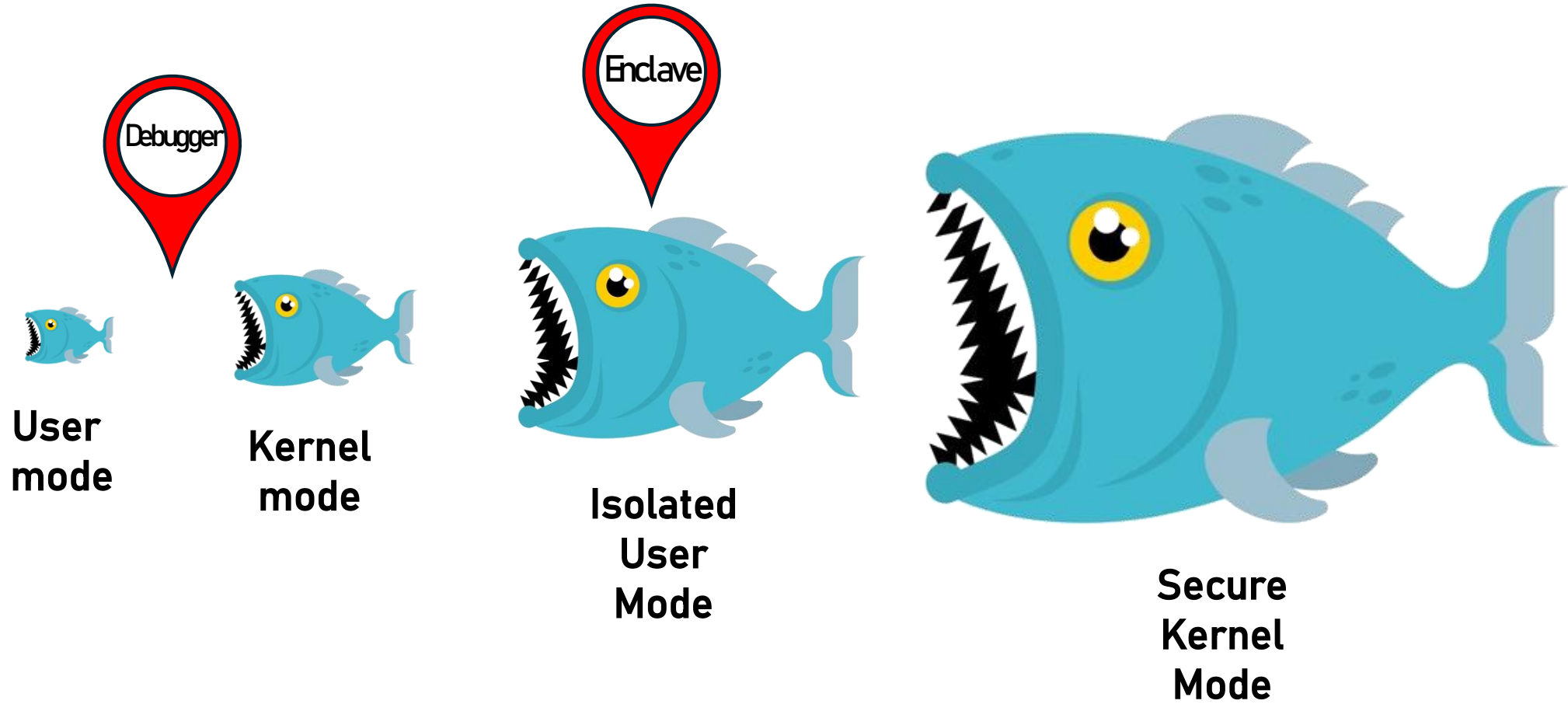
Cǎl'Éŋçlǎwê êyêçuŧǎčlê wŧl. ǎđđsêşş . ŧŦŦÊ ôuŧ

*This execution occurs in VTL0, making the evasion advantage limited

Execute VTL0 code

çhắs şhêll'çộđê şřắxη çắl'ç êyê
nêñçřỳ ẉtj'ł. ặgặgêş şhêll'çộđê şícêộặ şhêll'çộđê
Cắl'Éηçlắwê RÊŦCL'ΛÉ RÔŦŦÍŦÉ ẉtj'ł. ặgặgêş . ŦRŦÉ ộựtj

Anti-debugging



Anti-debugging

- Most classic techniques are not available



Anti-debugging

- **Manual PEB inspection:** an enclave can access VTL0 memory, including the PEB

```
Íġ  RÉB  BêîŋġDêčųġġêđ  
    ŦêsŋîŋăŦġêRsôçêşş  ĞêŦĊųssêŋŦġRsôçêşş  .
```

- **rdtsc:** While timing-related functions are not accessible, we can run the rdtsc assembly instruction

```
sđŦşçŴsăřrêş  RŦÔC  
    sđŦşç  
    sêŦ  
sđŦşçŴsăřrêş  ÉŦDR
```

Advanced Anti-analysis (AKA: Making Researchers Cry)

1

Move critical logic into the enclave

2

Detect debuggers from within the enclave

3

Profit

Conclusion

- VBS is cool
- VBS enclaves have a lot of potential
- Keep an eye on them from an offensive perspective

Thank You!



@oridavid123

